

Diario de Desarrollo: Sistema RAG para Verificación de Noticias

3/02 - Configuración de Infraestructura (Elasticsearch)	2
04/02 - Fase 2: Validación del Sistema de Recuperación (Retrieval)	4
07/02 - Fase 1 (Continuación): Auditoría Forense de Datos y Optimización "Quirúrgica" del ETL	6
17/02 - Fase 3: Despliegue del Modelo SLM e Integración RAG Distribuida	7
18/02 - Fase 4: Reestructuración de Entornos y Metodología de Evaluación Automática	8
10/03 - Implementación embeddings	9
12/03 - Optimización de Evaluación RAG y Dataset Final	11
17/03 - Refinamiento del Sistema de Evaluación y Flujo de Verificación Mixto	11
26/03 - Incorporación de Grupo de Control y Planificación de la Fase Final de Evaluación	12
14/04 - Ejecución de la Evaluación a Gran Escala y Consolidación de la Comparativa a 5 Bandas	13

3/02 - Configuración de Infraestructura (Elasticsearch)

1. Despliegue del Motor de Búsqueda (Elasticsearch)

Para cumplir con el objetivo de instalar la base de datos en entorno CPU sin permisos de root, se ha optado por una instalación portable (local) de Elasticsearch.

Comandos de instalación ejecutados:

1. Descarga del binario (Versión 8.12.0)

```
wget https://artifacts.elastic.co/downloads/elasticsearch/elasticsearch-8.12.0-linux-x86_64.tar.gz
```

2. Descompresión

```
tar -xzf elasticsearch-8.12.0-linux-x86_64.tar.gz
```

3. Acceso al directorio

```
cd elasticsearch-8.12.0/
```

2. Arranque del Servicio

Se ejecuta en modo daemon (segundo plano) para no bloquear la terminal. Se deshabilita la seguridad (X-Pack) para desarrollo y se cambia el puerto por defecto (9200) al **9250** para evitar conflictos con otros usuarios del servidor.

Comando de inicio en el directorio de elasticsearch:

```
./bin/elasticsearch -d -E xpack.security.enabled=false -E discovery.type=single-node -E http.port=9250
```

Comando de verificación:

```
curl http://localhost:9250
```

Debe devolver un JSON con el tagline "You Know, for Search"

3. Gestión del Proceso (Apagado)

Al no ser un servicio del sistema, el proceso persiste tras cerrar sesión si no se detiene manualmente.

Procedimiento de parada:

1. Localizar el proceso: `ps -ef | grep elasticsearch`
2. Matar el proceso por PID: `kill [PID]`
3. Alternativa rápida: `pkill -f elasticsearch`

Fase 1: Ingeniería de Datos (Pipeline ETL)

1. Desarrollo del Script de Ingesta (1_ingesta_datos.ipynb)

Se ha implementado un **Pipeline ETL (Extract, Transform, Load)** en Python utilizando Jupyter Notebook para poblar la base de datos vectorial.

- **Extracción (Extract):** Lectura del dataset comprimido (.gz) mediante streaming (generadores de Python) para procesar línea a línea y evitar desbordamientos de memoria RAM dado el volumen de datos (~800k líneas).
- **Transformación (Transform):**
 - **Limpieza:** Creación de función `clean_text` para decodificar entidades HTML (ej: `í $\rightarrow$ i`) y eliminar caracteres corruptos.
 - **Normalización:** Estandarización de campos en estructura JSON (title, body, date, url, source).
 - **Decisión Arquitectónica (Omisión de Chunking):** Se tomó la decisión deliberada de no fragmentar los textos (*chunking*) antes de la indexación. Dado que el corpus está compuesto por noticias periodísticas (textos inherentemente cortos y autoconclusivos) y el modelo generativo seleccionado para fases posteriores (Phi-3-mini-4k) posee una ventana de contexto amplia (4.096 tokens), la inyección del documento completo (top-k=1) es viable. Esto preserva la coherencia narrativa, mantiene unidos los metadatos (título, fecha) al cuerpo de la noticia, y optimiza el rendimiento del algoritmo de recuperación léxica (BM25), evitando la pérdida de contexto que generan los fragmentos huérfanos.
- **Carga (Load):** Indexación masiva en Elasticsearch utilizando la API `helpers.bulk` para optimizar el rendimiento de red.

2. Configuración del Índice (noticias_tfg)

Se ha definido un mapping explícito en Elasticsearch para optimizar la recuperación de información:

- **Analizador:** Se configuró analyzer: "spanish" en los campos de texto (title, body) para que el motor entienda conjugaciones y plurales en español (Stemming).
- **Robustez:** Se activaron parámetros como `ignore_malformed: True` en fechas y `ignore_above: 256` en URLs para asegurar que datos atípicos no detengan el proceso de indexación.

3. Resultados de la Ejecución

El proceso de ingesta se completó exitosamente validando la conexión al puerto **9250**.

- **Tiempo de ejecución:** ~46 segundos.
- **Documentos indexados (Éxitos): 521.816** noticias disponibles para búsqueda.
- **Datos descartados:** El script filtró automáticamente las líneas corruptas o con formato JSON inválido (diferencia entre las ~800k líneas brutas y los ~520k documentos finales válidos), asegurando la calidad de la "Biblioteca".

04/02 - Fase 2: Validación del Sistema de Recuperación (Retrieval)

1. Refinamiento del Pipeline de Ingesta (Corrección de Metadatos)

Durante las primeras pruebas de inspección de datos, se detectó una inconsistencia en la extracción de la fuente de la noticia (campo source), que devolvía valor "Desconocido" en la totalidad de los documentos.

- **Diagnóstico:** El análisis de la estructura cruda (raw JSON) reveló que la información de calidad (publisher, articleBody) no residía en la raíz del objeto, sino anidada dentro de un bloque jsonld (JSON-LD para SEO).
- **Corrección:** Se modificó el script 1_ingesta_datos.ipynb implementando una lógica de extracción en cascada: buscar primero en jsonld y, si falla, buscar en la raíz.
- **Resultado:** Re-indexación completa exitosa. Ahora el sistema identifica correctamente medios como *El País*, *ABC*, *La Razón*, *El Universal*, etc.

2. Implementación de la Lógica de Búsqueda (02_Pruebas_Retrieval.ipynb)

Se ha desarrollado y validado el componente de recuperación (Retriever) utilizando la librería elasticsearch de Python. Se diseñó una **query DSL** específica para maximizar la relevancia del contexto que se pasará al LLM:

- **Estrategia Multi-Match:** La búsqueda se realiza simultáneamente sobre los campos title y body.
- **Boosting (Ponderación):** Se aplicó un *boost* de $\wedge 3$ al título ("fields": ["title $\wedge 3$ ", "body"]).
 - *Justificación:* Las palabras clave en el titular tienen mayor densidad semántica y relevancia temática que una mención aislada en el cuerpo del texto.
- **Tolerancia a Erratas (Fuzziness):** Se activó el parámetro fuzziness: "AUTO".
 - *Objetivo:* Permitir que el sistema encuentre entidades nombradas aunque el usuario cometa errores ortográficos (ej. buscar "Mbape" y recuperar "Mbappé").

3. Resultados de las Pruebas de Recuperación

Se ejecutaron tres casos de uso para validar la robustez del índice noticias_tfg (521.816 documentos):

1. **Búsqueda Factual ("*guerra comercial Trump aranceles China*"):**
 - **Score BM25:** Alto (~78.3).
 - **Precisión:** Los top 5 resultados son noticias de 2025 y finales de 2024 directamente relacionadas con el tema, validando la capacidad del sistema para recuperar contexto temporalmente relevante.
2. **Prueba de Robustez ("*fichaje Kylian Mbape Real Madrid*"):**
 - **Resultado:** El sistema corrigió automáticamente "Mbape" a "Mbappé".
 - **Relevancia:** Recuperó noticias de fuentes fiables (*RTVE*, *El Confidencial*) confirmando el evento, demostrando que el *Fuzzy Search* funciona correctamente.
3. **Fuentes:** Tras la corrección del paso 1, los resultados muestran explícitamente el origen de la información, lo cual será vital para la fase de verificación de veracidad.

4. Definición del Flujo de Datos para la Fase de Generación (Puente a Fase 4)

Tras validar la recuperación visual, se ha establecido el diseño técnico necesario para conectar la base de datos con el LLM en la siguiente etapa.

Diferenciación de Funciones:

- **Estado Actual (Validación):** La función `retrieve_context` diseñada en esta fase tiene una finalidad de **auditoría humana**; imprime los resultados en pantalla (`print`) para verificar la relevancia y las fechas.
- **Estado Futuro (Producción):** Para la Fase 4, esta función se refactorizará (`get_context_for_llm`). El objetivo dejará de ser visual y pasará a ser funcional: deberá **retornar (return)** una cadena de texto única que concatene el contenido de las noticias recuperadas.

Diseño del "Mega-Prompt" (Context Injection): Se ha clarificado el flujo lógico del sistema RAG. El LLM no consultará Elasticsearch directamente. El script de Python actuará como intermediario ("Pegamento") realizando los siguientes pasos:

1. **Retrieval:** Elasticsearch devuelve los 5 documentos más relevantes (`top_k=5`).
2. **Concatenación:** Python extrae el texto limpio del cuerpo de esas 5 noticias (posible gracias a la corrección del campo `articleBody` en la fase ETL) y construye un bloque de texto unificado.
3. **Prompt Engineering:** Se inyectará este bloque dentro de un prompt estructurado junto a la pregunta del usuario.

Esquema lógico del Prompt planificado:

"Eres un asistente verificador. Responde a la pregunta: [PREGUNTA] basándote ÚNICAMENTE en el siguiente contexto recuperado: [NOTICIA 1] + [NOTICIA 2] + [NOTICIA 3]..."

Esta validación confirma que los datos tienen la calidad y estructura necesarias para que el modelo de lenguaje pueda "leerlos" y generar respuestas fundamentadas, cumpliendo el objetivo de mitigar alucinaciones mediante una fuente de verdad externa.

07/02 - Fase 1 (Continuación): Auditoría Forense de Datos y Optimización "Quirúrgica" del ETL

1. Auditoría de Estructura de Datos (Data Profiling)

Antes de dar por cerrada la ingesta, se planteó la necesidad de verificar empíricamente la heterogeneidad de los datos para no basar el código en suposiciones. Se creó un script de "auditoría forense" en el **Notebook 01** para analizar una muestra masiva (800k líneas) y determinar la ubicación exacta del contenido relevante.

- **Hallazgos de la Auditoría:**
 - **Fuentes (publisher):** El 100% de las fuentes identificables residen dentro del objeto jsonld (estándar Schema.org) como diccionarios. No se encontraron listas ni datos en la raíz.
 - **Cuerpos (articleBody):** El contenido textual reside exclusivamente en jsonld. La cantidad de cuerpos de noticia en la raíz del JSON fue **0**.
 - **Contenido Multimedia:** Se detectó un subconjunto de noticias (aprox. 200k) que no tenían articleBody pero sí description dentro de jsonld (correspondientes a vídeos de RTVE o galerías).
 - **Registros Vacíos:** Se identificaron ~30.860 registros que no contenían ni texto ni descripción (ruido).

2. Refactorización del Script de Ingesta (Versión "Estricta")

Basándose en los datos de la auditoría, se reescribió la función `generate_actions` (versión V5) para eliminar deuda técnica y lógica innecesaria.

- **Lógica de Extracción Simplificada:** Se eliminaron todas las comprobaciones redundantes que buscaban datos en la raíz (`doc.get("articleBody")`), ya que la auditoría confirmó que eran inútiles. El script ahora apunta directamente a jsonld.
- **Estrategia de Fallback (Rescate de Datos):** Se implementó una lógica condicional para el contenido:
 - *Prioridad 1:* Extraer articleBody (Noticia estándar).
 - *Prioridad 2:* Si es nulo, extraer description (Para no perder noticias de vídeo/galerías).
- **Filtrado de Calidad (Data Cleaning):** Se añadió una regla de exclusión explícita (`continue`) para las ~30.000 noticias detectadas como vacías.

3. Resultado Final del Índice (noticias_tfg)

Tras la ejecución de la carga optimizada, se ha generado un índice más compacto y de mayor calidad semántica.

- **Volumen Final:** ~490.000 documentos (Se redujo desde los 521k originales al eliminar los 30k vacíos que solo aportaban ruido).
- **Cobertura:** Ahora se incluyen correctamente resúmenes de noticias multimedia que antes aparecían con el cuerpo vacío (...).

- **Eficiencia:** El proceso de ingesta es más rápido al realizar menos comprobaciones por documento.

4. Cierre de Fase de Ingeniería de Datos

Con la base de datos validada, limpia y optimizada, se procede al cierre de la conexión con el servidor de CPU (valencia) para migrar el desarrollo al servidor con GPU (valencia2). El objetivo es iniciar la **Fase 3: Generación**, donde se desplegará el modelo de lenguaje (LLM) que consumirá estos datos.

17/02 - Fase 3: Despliegue del Modelo SLM e Integración RAG Distribuida

1. Configuración del Entorno GPU y Conectividad Distribuida

Para iniciar la fase de generación, se ha migrado el desarrollo al servidor **valencia2 (GPU)**. El principal reto ha sido conectar el "Cerebro" (LLM en GPU) con la "Memoria" (Elasticsearch en CPU/valencia).

- **Infraestructura:** Se ha mantenido la base de datos en el servidor original para optimizar el almacenamiento, ya que el disco de valencia2 se encuentra al 97% de su capacidad.
- **Túnel SSH (Bridge de Red):** Debido a las restricciones de firewall entre servidores, se ha implementado un túnel SSH para mapear el puerto 9250 de valencia al localhost de valencia2.
 - **Comando clave (Replicable):** `ssh -L 9250:localhost:9250 javierruiz@valencia.inf.um.es`
- **Sincronización de Versiones:** Se detectó un conflicto de comunicación entre la librería `elasticsearch-py` (v9.x) y el servidor (v8.12.0). Se procedió a realizar un *downgrade* estricto de la librería en el entorno de la GPU para garantizar la compatibilidad total y el funcionamiento del comando `es.ping()`.

2. Carga del Modelo de Lenguaje (SLM)

Se ha seleccionado **Microsoft Phi-3-mini-4k-instruct** como modelo base para el sistema.

- **Optimización de Memoria:** El modelo se carga en precisión de punto flotante de 16 bits (`torch.float16`) utilizando `device_map="auto"` para aprovechar la GPU NVIDIA GeForce RTX 4090.
- **Estabilidad:** Se desactivó el parámetro `trust_remote_code` para utilizar la implementación nativa de la librería `transformers`, evitando errores de configuración en la arquitectura del modelo.

3. Refactorización del Pipeline RAG (Configuración "Profesor")

Siguiendo las directrices académicas para mitigar alucinaciones y permitir una evaluación limpia, se ha refactorizado la función `ask_rag` (evolución de `get_context_for_llm`).

- **Parámetros de Recuperación (Retrieval):**
 - **Top-K (\$k=1\$):** El sistema ahora recupera únicamente la noticia más relevante para forzar un "match perfecto" y facilitar la auditoría de la respuesta.
 - **Fuzziness (0):** Se ha eliminado la tolerancia a erratas para asegurar que la recuperación sea exacta y no basada en similitudes débiles.

- **Ponderación:** Se ha aumentado el boost del título a 5 para priorizar la coincidencia temática directa.
- **Parámetros de Generación (Inferencia):**
 - **Determinismo:** Se ha configurado `do_sample=False` y `temperature=0.0`. Esto elimina la aleatoriedad del modelo, obligándolo a ser literal con el texto recuperado.
 - **Prompt de Verificación:** Se ha diseñado un prompt de "Fact-Checking" que instruye al modelo a declarar ignorancia ("No tengo información suficiente") si la respuesta no está explícitamente en el texto de referencia.

4. Resultados de la Comparativa (Validación)

Se realizó una prueba comparativa con la consulta: "*¿Qué aranceles puso Trump a China?*":

1. **SLM Base (Sin RAG):** El modelo alucina mencionando leyes inexistentes como la "Ley de Protección de la Industria Automotriz" basada en su conocimiento de entrenamiento desfasado.
2. **Sistema RAG:** El modelo responde con precisión quirúrgica basándose en la noticia recuperada de *Excélsior* (Abril 2025), citando el arancel del 145% (125% adicional + 20% previo).

5. Notas para Próximas Sesiones

- **Persistencia:** Antes de trabajar en `valencia2`, siempre debe verificarse que Elasticsearch esté corriendo en `valencia` y que el túnel SSH esté activo.
- **Batería de Pruebas:** El siguiente paso es generar un conjunto de preguntas "trampa" (temas no presentes en el índice) para estresar la capacidad del sistema de evitar alucinaciones.

18/02 - Fase 4: Reestructuración de Entornos y Metodología de Evaluación Automática

1. Reestructuración y Cierre del Notebook 03 (Validación Técnica)

Para mantener la limpieza del proyecto y separar conceptos, se ha decidido que el Notebook 03 quede exclusivamente dedicado al despliegue de la infraestructura y validación del pipeline RAG.

- **Limpieza de Salida (Output Parsing):** Se ha refactorizado la función `ask_rag` para que no devuelva el historial de chat crudo. Ahora extrae la respuesta limpia navegando por la estructura de diccionarios (`outputs[0]['generated_text'][-1]['content']`) y retorna un objeto estructurado (Título, Contenido y Respuesta). Esto es vital para no inyectar "basura JSON" en los futuros archivos de resultados.

2. Diseño de la Batería de Preguntas (El "Ground Truth")

Se ha planificado la creación de un dataset propio de evaluación en formato CSV (`bateria_preguntas_tfg.csv`).

- **Estructura:** Contará únicamente con las columnas ID, Tipo (Factual, Trampa, Inferencia), Pregunta y Respuesta_Esperada (Ground Truth).

- **Evolución Metodológica:** Se ha descartado el uso de "Keywords" estáticas para la evaluación, ya que un enfoque basado en coincidencias exactas de palabras (String matching) penalizaría respuestas correctas formuladas de otra manera (ej. "145%" vs "125% más el 20% previo").

3. Planificación del Notebook 04 (Experimentación y Comparativa)

Se creará un nuevo entorno limpio (Notebook 04) dedicado a la ingesta del CSV de preguntas y la generación automatizada de resultados. El script iterará sobre la batería de preguntas y creará una columna de respuesta para cada una de las 4 arquitecturas a comparar:

1. **SLM Base (Sin RAG):** Para medir la alucinación pura del modelo local.
2. **LLM Base (Sin RAG):** Para evaluar si un modelo de mayores dimensiones sufre del mismo problema de conocimiento desfasado.
3. **RAG Simple:** SLM con inyección de contexto pero sin instrucciones restrictivas.
4. **RAG Complejo:** SLM con el prompt estricto de Fact-Checking diseñado en la fase anterior.

4. Sistema de Evaluación Semántica ("LLM-as-a-Judge")

Para resolver el problema de la evaluación automática, se implementará la técnica de "El LLM como Juez". En el Notebook 04, un modelo de lenguaje actuará como evaluador.

- **Funcionamiento:** Leerá la *Respuesta Esperada* de la batería y la comparará semánticamente con la *Respuesta Generada* por cada una de las 4 arquitecturas.
- **Ventaja:** El juez automático comprenderá el contexto, las sumas matemáticas implícitas y los sinónimos, asignando de forma autónoma una puntuación binaria (1 = Acierto, 0 = Fallo/Alucinación) a cada celda, generando un CSV final listo para la extracción de métricas y gráficas del TFG.

10/03 - Implementación embeddings

Fase 1b-3b: El salto a la Búsqueda Semántica (Embeddings) y Preparación del RAG Dual

Tras consolidar la arquitectura base con el buscador léxico tradicional (BM25), hoy he implementado el cambio de paradigma fundamental del proyecto: la **búsqueda semántica mediante embeddings**.

Mientras que BM25 busca coincidencias exactas de caracteres (si busco "rebaja", no encuentra "oferta"), los modelos de embeddings convierten el texto en vectores matemáticos (matrices de números). Esto permite a la base de datos medir la "distancia espacial" entre conceptos, entendiendo sinónimos y contexto.

Para mantener la coherencia con mi decisión de **no hacer text chunking** (dividir las noticias en fragmentos pequeños), he descartado modelos estándar con límite de 512 tokens y he seleccionado **BAAI/bge-m3**. Este modelo multilingüe de código abierto admite hasta 8192 tokens, lo que garantiza que ninguna noticia se trunque al vectorizarla.

Para implementar este nuevo "motor", he replicado la estructura de los notebooks originales creando una ruta paralela (la serie "b"):

1. Notebook 01b: Ingesta de Datos Vectorial

- **Proceso:** He vuelto a procesar las 490.956 noticias. Esta vez, he concatenado el campo title y body para darle el máximo contexto al modelo.
- **Almacenamiento:** Los vectores se enviaron a través del túnel SSH a un nuevo índice en Elasticsearch (noticias_tfg_vectores) configurado específicamente para búsquedas de Similitud del Coseno. La ingesta masiva (helpers.bulk) se configuró en lotes de 400 documentos para no saturar la red, tardando aproximadamente 3 horas con 0 fallos.

2. Notebook 02b: Pruebas de Retrieval Vectorial (kNN)

- **Proceso:** Validé matemáticamente que Elasticsearch recuperaba las noticias correctas mediante algoritmos k-Nearest Neighbors (kNN).
- **Resultados:** Se comprobó empíricamente la superioridad semántica en consultas complejas. Por ejemplo, al preguntar por una "rebaja" en aceite "Suroлива", el sistema recuperó perfectamente una noticia que hablaba de una "oferta nunca vista", algo en lo que el sistema BM25 original habría fallado.

3. Notebook 03b: Generación RAG Vectorial y Evaluación

- **Proceso:** He integrado la recuperación vectorial con el LLM **Phi-3-mini-4k-instruct**.
- **Corrección de Arquitectura:** Inicialmente intenté ampliar el contexto proporcionado al LLM subiendo a top_k=3. Esto provocó un colapso en la ventana de contexto de Phi-3 (fenómeno *Lost in the Middle* y alucinaciones). Para solucionarlo, he revertido la configuración a **top_k=2**

Próximos pasos: El Notebook 04 (El Enfrentamiento)

Con las dos vías del RAG terminadas y encapsuladas de forma idéntica en los Notebooks 03 (BM25) y 03b (Vectorial), el siguiente paso lógico es crear el **Notebook 04**.

En esta futura fase, el objetivo será enfrentar ambos sistemas. Diseñaré un script que recorra automáticamente mi batería de preguntas de prueba, lanzando cada pregunta tanto al RAG Léxico como al RAG Semántico, y guardando sus respuestas estructuradas para poder compararlas de forma objetiva y sacar las conclusiones finales del TFG.

12/03 - Optimización de Evaluación RAG y Dataset Final

- **Refinamiento del Dataset de Evaluación:** Se han generado y estructurado las entradas **ID 57 a 80**, cubriendo temas nuevos (tractoradas en España, Premios Oscar 2025, elecciones en Reino Unido, Apple Vision Pro y el eclipse solar 2024). Se ha verificado que cada pregunta tenga un dato factual único y extraíble para evitar alucinaciones.

17/03 - Refinamiento del Sistema de Evaluación y Flujo de Verificación Mixto

- **Optimización del *Prompt* del Juez:** Se detectaron sesgos en la evaluación automática, incluyendo falsos positivos (por exceso de verbosidad en los modelos base) y falsos negativos (por excesiva rigidez sintáctica del juez ante respuestas largas que sí contenían el dato). Se reescribió el *System Prompt* para aplicar una "Regla de Inclusión y Flexibilidad Semántica" combinada con una "Prioridad del Dato" estricta, obligando al juez a centrarse únicamente en la presencia exacta de la información clave (cifras, fechas, nombres) y castigando severamente las alucinaciones.
- **Mitigación del Sesgo de Autopreferencia (*Self-Preference Bias*):** Tras evaluar las cuotas y modelos disponibles en la consola de Groq, se decidió cambiar el modelo utilizado para el rol de evaluador (*LLM-as-a-judge*). Evaluar las respuestas de la familia Llama 3 con otro modelo de la misma familia comprometía la neutralidad del experimento. Se configuró el uso de modelos alternativos para garantizar una evaluación ciega y libre de sesgos arquitectónicos.
- **Nuevo Flujo de Evaluación (Híbrido Automático-Manual):** Se ha establecido la metodología definitiva para la evaluación final. El modelo Juez actuará como una **primera capa de cribado automatizado**, realizando la comparación inicial y etiquetando en masa. No obstante, para garantizar un rigor científico del 100% en los resultados del TFG, **se implementará una fase de auditoría manual posterior**. En esta fase se revisarán minuciosamente todas las salidas, corrigiendo a mano cualquier etiqueta de "CORRECTO" o "INCORRECTO" que el juez haya podido clasificar erróneamente debido a las limitaciones intrínsecas de los LLMs como evaluadores.

26/03 - Incorporación de Grupo de Control y Planificación de la Fase Final de Evaluación

1. Establecimiento del Grupo de Control (Notebook 05)

Tras la exitosa evaluación de la prueba de concepto inicial con 80 preguntas, se detectó la necesidad metodológica de descartar posibles sesgos arquitectónicos en los resultados. Para demostrar que las deficiencias en la recuperación de datos factuales (alucinaciones) son inherentes a los Modelos de Lenguaje Grande (LLMs) operando sin contexto externo, se ha procedido a:

- **Integración de un Segundo LLM Avanzado:** Se ha incorporado el modelo Qwen 3 (32B) a través de la API de Groq como grupo de control operando en *Zero-Shot*. Al poseer una arquitectura radicalmente distinta a la familia Llama, garantiza una comparativa justa y ciega frente al modelo Llama 3.3 70B evaluado previamente.
- **Aislamiento del Entorno de Ejecución:** Para evitar el desbordamiento de memoria en la tarjeta gráfica local (errores OOM en la VRAM) experimentados al acumular tensores, esta nueva evaluación se ha encapsulado en un entorno independiente (Notebook 05).
- **Ampliación del Dataset sin Reevaluación:** Las nuevas respuestas generadas por Qwen 3 han sido sometidas al escrutinio del LLM-Juez estricto (openai/gpt-oss-120b). La columna de resultados y correcciones se ha insertado directamente en el CSV existente (resultados_evaluacion_80_preguntas.csv), preservando intactas las auditorías manuales previas de los otros 4 casos y permitiendo generar un gráfico comparativo a 5 bandas.

2. Planificación de la Fase Final del Proyecto (Notebook 06)

Con el entorno de evaluación estabilizado a 5 bandas (SLM Base, LLM A, LLM B, RAG Léxico y RAG Semántico), se ha definido la hoja de ruta metodológica para la culminación del Trabajo de Fin de Grado en el futuro Notebook 06:

- **Escalado del Dataset (Ground Truth):** La batería de validación se ampliará conservando las 80 preguntas iniciales y añadiendo 120 preguntas nuevas, alcanzando un volumen final de 200 iteraciones para dotar al estudio de mayor robustez estadística.
- **Evaluación Integral:** Las 120 nuevas cuestiones serán procesadas y evaluadas automáticamente por las 5 configuraciones establecidas, consolidando un archivo global denominado resultados_evaluacion_5_modelos.csv.
- **Análisis de Componentes Óptimos:** Se analizarán empíricamente los resultados sobre la muestra completa de 200 preguntas para dictaminar dos métricas clave basándose en el volumen de aciertos:
 1. Cuál es el LLM que mejor desempeño ofrece como motor de razonamiento.
 2. Cuál es el motor de recuperación más preciso (BM25 vs. Vectorial).
- **Fusión y Validación Definitiva:** Se extraerán los componentes ganadores del análisis anterior para ensamblar la arquitectura RAG definitiva (Mejor LLM + Mejor Retriever). Este sistema final procesará nuevamente las 200 preguntas. El rendimiento definitivo se volcará en el dataset resultados_RAG_final.csv, cuyo análisis culminará con la generación de un histograma final de Aciertos vs. Fallos, cerrando así el ciclo empírico del proyecto.

14/04 - Ejecución de la Evaluación a Gran Escala y Consolidación de la Comparativa a 5 Bandas

Tras la fase de planificación y el establecimiento del grupo de control, se ha procedido a la ejecución masiva y definitiva de las pruebas mediante el nuevo entorno de validación (Notebook 06). Los hitos alcanzados en esta sesión son los siguientes:

1. Escalado del Dataset (Ground Truth): Superando las estimaciones iniciales de la fase de planificación (200 iteraciones), se ha ampliado la batería de validación inyectando **140 preguntas nuevas**, que sumadas a las 80 originales ya auditadas, conforman un *dataset* definitivo de **220 iteraciones**. Este incremento dota al estudio de una robustez estadística aún mayor para la toma de decisiones finales.

2. Ejecución Compartimentada y Gestión de Recursos (Notebook 06): Para garantizar la estabilidad del sistema, optimizar las llamadas a las APIs y asegurar la integridad de los datos, la arquitectura de evaluación se ha diseñado en dos fases secuenciales y aisladas, implementando puntos de control (*checkpoints*) con codificación multiplataforma (utf-8-sig) para evitar corrupciones de formato (Mojibake):

- **Fase 1 (Evaluación Zero-Shot):** Se ha sometido a las 140 nuevas preguntas al conocimiento paramétrico estático de los tres modelos base (Phi-3 Local, Llama 3.3 70B y Qwen 3 32B).
- **Fase 2 (Evaluación RAG):** De forma independiente, se han activado los motores de recuperación (Léxico BM25 y Semántico BGE-M3) para inyectar contexto dinámico en el modelo local (Phi-3), evaluando su capacidad de generación fundamentada.

3. Tribunal Automático y Consolidación de Datos: Las respuestas generadas por los 5 sistemas en las 140 iteraciones nuevas han sido sometidas al escrutinio del modelo Juez (openai/gpt-oss-120b). Tras la limpieza y estandarización algorítmica de sus veredictos (reduciéndolos a variables binarias CORRECTO/INCORRECTO), los resultados parciales se han concatenado verticalmente con el historial auditado de las 80 preguntas iniciales. El resultado es la creación de la joya de la corona de la fase analítica: el archivo global **resultados_evaluacion_5_modelos.csv**.

4. Diagnóstico de Rendimiento Visual: Como culminación de esta fase, se ha generado el gráfico comparativo final (grafico_rendimiento_220_preguntas.png). Este diagrama de barras horizontales expone de forma clara y empírica el porcentaje de aciertos real de cada una de las 5 configuraciones sobre el total de 220 preguntas.

Próximos pasos: Con los datos masivos ya recolectados y visualizados, se procederá al análisis de los resultados para extraer los componentes ganadores (el mejor LLM y el mejor motor de recuperación) y ensamblar la arquitectura RAG definitiva que cerrará el ciclo del proyecto.

19/04 - Ensamblaje, Ejecución y Evaluación Definitiva del Sistema RAG Óptimo

Como culminación de la fase analítica, se ha procedido a la selección de los componentes con mejor desempeño y al ensamblaje de la arquitectura final del proyecto mediante el desarrollo de un nuevo entorno de ejecución (Notebook 07). Los procedimientos y resultados de esta sesión son los siguientes:

1. Selección Empírica de Componentes: A partir del análisis de los datos consolidados en la fase anterior, se determinó objetivamente la configuración óptima para el sistema final. Se seleccionó el modelo de lenguaje **Llama 3.3 70B** como motor de razonamiento (por su superioridad en la comprensión y síntesis de información) y el **RAG Semántico** (basado en vectores con el modelo BGE-M3) como motor de recuperación de contexto.

2. Integración de la Arquitectura Definitiva (Notebook 07): Se desarrolló el entorno de evaluación final acoplando los componentes seleccionados. Para eludir las limitaciones previas de memoria VRAM, la generación de lenguaje se externalizó mediante la API de Groq, mientras que la generación de *embeddings* (BGE-M3) y la búsqueda en Elasticsearch se mantuvieron en la infraestructura local procesándose a través de la CPU. Se reutilizaron exactamente los mismos *prompts* sistémicos de las fases anteriores para garantizar el rigor y la comparabilidad metodológica.

3. Procesamiento a Gran Escala y Depuración de Datos: El sistema RAG definitivo procesó de forma ininterrumpida la batería completa de 220 preguntas. Durante la ejecución y el guardado incremental, se identificaron y solventaron anomalías estructurales en la delimitación del archivo CSV (fusiones de registros causadas por la pérdida de saltos de línea al interrumpir la ejecución). Se aplicaron rutinas de saneamiento de datos mediante expresiones regulares (re), consolidando la integridad estructural en el archivo definitivo resultados_RAG_Final_TFG_LISTO.csv.

4. Validación Automatizada Continua: Las 220 respuestas generadas por el sistema definitivo fueron sometidas a la evaluación binaria (CORRECTO/INCORRECTO) del modelo Juez estricto (openai/gpt-oss-120b). Se mantuvo inalterado el protocolo de validación para asegurar que el rendimiento medido fuera directamente comparable con los resultados del grupo de control y de los modelos base.

5. Generación de la Visualización Global: Como cierre de la fase empírica, se programó un *script* de visualización avanzada empleando la biblioteca *Seaborn*. El resultado fue la exportación del documento grafico_maestro_tfg.png, un diagrama de barras horizontales consolidado que superpone la tasa de éxito absoluta del sistema RAG final frente a las cinco configuraciones previamente evaluadas, permitiendo observar de manera directa el impacto cuantitativo de la arquitectura propuesta.